Lab 2 Report

Point-to-Point Communication in MPI

Huzaifa Arshad
Roll No: 2023-CS-86
Department of Computer Science
University of Engineering and Technology Lahore

March 3, 2026

1 Objective

The objective of this lab was to understand the MPI execution model and implement blocking point-to-point communication using `MPI_Send` and `MPI_Recv`. Execution time was measured and speedup was analyzed. Wildcard communication using `MPI_ANY_SOURCE` was also investigated.

2 Exercise 1: Parallel Sum with Timing and Speedup

2.1 Description

Each MPI process computes a partial sum of numbers from 1 to 1000. Process 0 collects partial sums from all worker processes and calculates the final result.

2.2 Execution Results

Processes	Execution Time (seconds)	Speedup
1	0.000001	1.00
2	0.000065	0.015
4	0.000018	0.055

Table 1: Execution Time and Speedup

Speedup is calculated as:

$$Speedup = \frac{T(1)}{T(N)}$$

2.3 Analysis

The speedup is not linear because:

- Communication overhead dominates computation.
- The problem size (1000 numbers) is very small.
- Synchronization using `MPI_Barrier` adds overhead.
- According to Amdahl's Law, parallel performance is limited by the sequential portion.

The system did not support more than 4 processes due to limited available slots.

3 Exercise 2: Custom Gather Using MPI_Send and MPI_Recv

3.1 Description

Each worker process (rank 1 to size-1) generates an array of 10 integers and sends it to process 0 using point-to-point communication. Process 0 receives and prints the arrays.

3.2 Output

```
Process 0 received from 1: 100 101 102 103 104 105 106 107 108 109
Process 0 received from 2: 200 201 202 203 204 205 206 207 208 209
Process 0 received from 3: 300 301 302 303 304 305 306 307 308 309
```

3.3 Source Code

```c
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[]) {

    int rank, size;
    MPI_Status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int arr[10];

    if (rank != 0) {

        for (int i = 0; i < 10; i++)
            arr[i] = rank * 100 + i;

        MPI_Send(arr, 10, MPI_INT, 0, rank, MPI_COMM_WORLD);
    }
    else {

        for (int i = 1; i < size; i++) {

            MPI_Recv(arr, 10, MPI_INT, i,
                     MPI_ANY_TAG,
                     MPI_COMM_WORLD, &status);

            printf("Process 0 received from %d: ",
                   status.MPI_SOURCE);

            for (int j = 0; j < 10; j++)
                printf("%d ", arr[j]);

            printf("\n");
        }
    }

    MPI_Finalize();
    return 0;
}
```

4 Exercise 3: Investigation of Wildcards

4.1 Modification

Process 0 was modified to use wildcard receive:

```
MPI_Recv(arr, 10, MPI_INT,
         MPI_ANY_SOURCE,
         MPI_ANY_TAG,
         MPI_COMM_WORLD,
         &status);
```

4.2 Observed Output

Multiple executions produced different receive orders:

```
Run 1:
Received from process 2
Received from process 3
Received from process 1

Run 2:
Received from process 3
Received from process 1
Received from process 2

Run 3:
Received from process 1
Received from process 2
Received from process 3
```

4.3 Explanation

When using `MPI_ANY_SOURCE`, process 0 receives messages in the order they arrive. Since all worker processes execute concurrently, arrival order depends on operating system scheduling and communication timing. Therefore, the receive order is not deterministic.

5 Reflection

Point-to-point communication requires explicit coordination between sender and receiver processes. Incorrect tag usage or mismatched ranks may cause deadlocks or program hangs. Blocking communication requires careful ordering to avoid cyclic dependencies. Compared to collective operations such as `MPI_Reduce`, point-to-point communication requires more manual coding but offers greater flexibility. This lab strengthened my understanding of MPI message matching rules, blocking behavior, wildcard usage, and communication overhead in parallel systems.